

lity Management team's priorities are a combination of team strategy, operational work criticality, commitments to customers and other GitLab teams and how urgently we need to deliver a body of work, and overall alignment with the results we have committed to deliver within a certain timeframe.

Sizing

A combination of issue weight and sizing labels such as `~vuln-mgmt::small` which are used to define the estimated engineering effort which will be required to close an issue, be it fixing a bug or delivering an outcome in line with team priorities.

Milestone

For the purposes of development in Vulnerability Management, we use GitLab release milestones to plan our work. When we plan a milestone, we plan from the start date to the end date of the milestone. The end date, for our purposes, is the end of the entire milestone period. In the GitLab development and release cycle, the end of the milestone period is followed by the release process, and then subsequently the GitLab release date. We do not use the release date as the end of the milestone date, but rather the actual end date listed in the linked release development milestones list.

Milestone Planning

The process of deciding which work will be planned for a given team milestone. Milestone planning is primarily managed by the Engineering Manager. Milestone planning is typically driven by team priorities and capacity.

Velocity

The measure of engineering work, based on standardised units used when sizing work (i.e. issue weight), which has been performed over a given repeatable interval. It is related to understanding team capacity.

Capacity

The capacity of a team is a general measure of how much work can reasonably be expected to be delivered over a specific time (for example, a milestone) based on team size, responsibilities, absence and other factors. It is typically calculated by observing velocity and increased by adding additional team members, or performing retrospectives and addressing any impediments to team velocity.

Abstraction

Abstractions in the context of this policy are treated as any code which is added so that existing code may be made more generic and remove repetition of logic.

Example of abstractions are:

- Modelling a higher-level more generic representation of data to generalize implementation-specific data such a data from an external system

- Removing similar logic in favour of a more generic implementation for logic which might have been repeated in several places, with minor differences

MVP (Minimum Viable Product)

An MVP describes the minimum implementation for an idea, feature, or fix. MVPs are scoped to deliver the smallest amount of change or new functionality in order to deliver or validate a goal or idea. The goal of an MVP is to prevent over-engineering whilst approaching a fix or feature during engineering work. MVPs are mostly applicable to work which is designed to solve a business problem with an engineering solution, and the minimum amount of engineering effort which can address the business problem as documented is considered to be the MVP. Building and iterating on MVPs and using MVPs to prove ideas and share results are in line with our Iteration, Results for Customers and Collaboration values.

MVC (Minimum Viable Change)

MVC (not to be confused with the Model View Controller application pattern). A minimum viable change is the smallest change we can make to code or infrastructure to solve a specific defined problem. Focusing on making the minimal viable change is in line with our Iteration and Results for Customers values, as it's a way to prevent over-engineering to achieve outcomes.

Experiments

Experiments are standalone, new and often short-lived projects with a known purpose and fixed goal. They are typically developed by a single team member to prove out an idea prior to investing further engineering effort in proper design and implementation. We do experiments to avoid sinking time into contributing new features and capabilities to our tooling and automation, without having first validated if they are as beneficial or effective as hoped.

Experiments are scoped strictly as MVPs. Once the MVP has been built and proved (or disproved) the idea, further work should be documented as engineering work and planned according to this standard.

An experiment, at most, should have a maximum weight of 5. If more time is needed (i.e. longer than a week) to prove the idea, it is likely a larger feature or complex problem space and needs to be treated as planned and scoped engineering work.

Experiments should be developed in the Vulnerability Management Internal project.

The internal project versus the public project is preferred for experiments as typically licensing considerations will not be approached for experiments until they are validated, at which point they may be made public if appropriate.

Maintainability

This is a fairly broad term, but in the context of software engineering it focuses on how easy it is to perform maintenance on code or infrastructure.

Some examples include:

- Managing software dependencies with a package manager, to make it easier to update and install dependencies.
- Managing infrastructure as Infrastructure as Code, to make easier to update and rebuild infrastructure, as well as facilitating review of changes
- Having documentation. By documenting implementation details and the issues we work on, and most importantly why they exist, we reduce the time spent in research and discovery before team members can contribute changes.
- Having appropriate tests. Testing code logic makes it much easier to validate changes and increases velocity and makes it easier to contribute code.
- Simple implementations, this can also apply to dependency selection. Perhaps you can use a simple HTTP library instead of using an entire MVC web application framework for a tool? The former may require much less of a learning curve to contribute to, which is good for maintainability.

Some examples of things which hurt maintainability are:

- Having tests which have highly complex environmental dependencies which team members can face difficulty contributing to and running, which slows down velocity and leads to lower test adoption.
- Forked libraries as dependencies, and bespoke implementations where well maintained community-maintained libraries could be used instead. These introduce maintenance burden which may be avoided with a different approach.
- Over-engineering of solutions and unjustified complexity. By focusing on delivering MVPs, we make code easier to iterate on.

Exceptions

There are a few exceptions to this policy. In cases where these exceptions apply, we should do our best to maintain this standard. To avoid blocking innovation and iteration though, the list below notes situations and which parts of the policy do not apply:

Experiments

Experiments are exempt from Code Review requirements. When developing experiments to validate new techniques or tools, you are exempt from the Code Review standards whilst iterating, as you are most likely working by yourself on the experiment. You are of course free to ask team members for help with problems and to review code, you are just not required to do so by this policy.

Reactive Work

Reactive Work often are not able to be anticipated, and occasionally will represent an emergency situation. As a result, addressing them is exempt from our Milestone Planning. When planning a milestone, we leave some capacity free to account for emergency or reactive work we can't anticipate.

Policy

Planning

Milestone Planning Meetings

Prior to a new milestone beginning, the weekly team sync call will be used as a milestone planning meeting.

The date of the milestone planning meeting will be the date of the team sync which has the fewest number of US business days between the team meeting and the start date of the next milestone. For example, if the milestone starts on a Wednesday, and the the team sync is on a Thursday, the Thursday following the the milestone start date would be the milestone planning meeting, rather than the prior Thursday.

When planning milestones, we use the start date and end date as showing in the GitLab release milestones on the main GitLab project. With the exception of any product work we do, we do not observe a feature freeze a week prior to the release date (the end of the milestone) so when we plan for a milestone, we plan from the beginning of the milestone through to the GitLab release milestone end date. Note, that whilst the release date follows the end date, the end date is not the same as the release date. This is consistent with the dates listed in the linked milestone list.

During this meeting, team members are expected to be ready to discuss issues for inclusion in the upcoming milestone.

This meeting is run by the Engineering Manager, as the responsible team member for the overall milestone plan. When they are not available, it is typical for a Staff Engineer to run the meeting, but this is not a requirement of this policy, another team member may be asked to step in due to availability reasons.

Prior to the meeting, a new milestone planning issue must be created, using the milestoneplanning.md issue template in the main Vulnerability Management tracker. This will be used to document the outcomes of the planning meeting and serve as a reference for team members.

During the meeting, the primary goals are to:

- Have the Engineering Manager (or stand-in) remind team members of current team priorities
- Have team members in turn discuss issues for inclusion in the milestone
- Review issues being discussed for inclusion based on this policy, and reach consensus on what to include
- Discuss any availability constraints (leave, illness, etc) and adjust allocations as appropriate
- Leave team members enough free capacity to handle the weekly reactive rotation, and the operational work involved (~5 points, ideally)
- Decide how to handle any work which needs to continue or has follow-up work from the previous milestone

Issue requirements for inclusion in a milestone

For an issue to be included in a milestone, it must:

- Be properly estimated (sizing and scoring)
- Be properly documented, per the Engineering Issue policy documented on this page

- Have an understandable proposed approach and ideally implementation documented
- Be able to be completed during the milestone
- Be aligned with current team, department, division and company goals and priorities

Outcomes

After a milestone is planned, each engineer in the team should have a clear view of the planned work they will be working on during the milestone. All appropriate issues will have their milestone set to the relevant current milestone. All issues must be assigned and have the appropriate milestone set. The milestone must represent all planned work which will be done in the milestone.

The outcomes of planning should be reflected in the milestone planning issue which was created prior to the planning meeting, and should include the planned work in the issue. This issue must be shared with the team and Product Security leadership for awareness.

Engineering Issues

How we document and work on issues in the Vulnerability Management team must follow this policy. This is high level guidance, but also a policy detailing expectations for issue quality. Issues which do not meeting this standard will typically not be considered to inclusion in a milestone, and therefore must not be worked on, noting the exceptions section of this page.

Engineering Issue Template

There is an issue template in the Vulnerability Management team's main tracker and the VulnMapper project issue tracker named Engineering Work. We aim to iterate on this template as these policies evolve, and where this template is in other projects we maintain we must add it as appropriate. It sets a simple format which makes it easier to remember the required information necessary for planning, documenting and justifying engineering work. It must be used when creating new engineering issues, and should be used as a guide when updating existing issues prior to milestone planning.

Sizing / scoring issues

As part of planning, we must size and score all issues with the appropriate labels and weights. These labels and our weighting methodology are documented in more detail on the Vulnerability Management team page, under "Planning" and the Development Labels page.

Priority

Issues will be labelled with a priority during regular priority adjustment & review performed by the Engineering Manager. They should not be added by Vulnerability Management team members unless there is specific guidance to add a priority to an issue, or if demonstrated business impact drives work which was not included in milestone planning.

Only issues included in a milestone or being included in a milestone outside of a milestone planning session should have a priority label. Priority labels may change during a milestone based on changing business or team commitments or priorities to better reflect work which

relates to focus areas with the most customer impact. These changes will be made as a result of a weekly review or as new information drives changes in priority.

In all cases, changes to a priority during a milestone must also include a comment, tagging the assigned engineer, giving background on the change in priority and making them aware of it, so they can adjust how they prioritize work for a given milestone.

We use the standard priority labels described in the Engineering Division Priority Triage handbook page and the Vulnerability Management Development Labels handbook page.

Documenting the why

An issue must explain why. For an issue which documents adding a feature to existing code, we need to document why the feature is needed. For an issue which changes existing code, we need to document why the change is necessary or desirable. For maintenance issues, we must also detail the why - although it is typically much shorter explanation.

For example: An issue updating the dependencies of a project could say "Why: To maintain a security software delivery lifecycle by demonstrating good practices we want to model for customers and our team")

For example: An issue describing a bugfix could say "Why: The issue linked to this issue describing the problem observed by other teams is being caused by a bug in this tool. We need to fix this bug to deliver promised functionality to the team/all team members/customers. This process is a critical part of compliance operations and not fixing this impacts compliance requirements."

For example: An issue describing a new capability could say "Why: We have had feedback from other teams that including this extra information on triage issues would save them 10 hours per week of manual triage work. Considering it is fairly easy to implement, adding the feature would improve overall company efficiency and make it easier to address security issues"

Documenting the problem or goal

This is an extension of the why. Whilst documenting the why documents the high level outcome and driver for some work, in the context of GitLab and our responsibilities to others, the problem or goal is a more detailed description of the change, fix or new functionality which needs to be added in the context of the tool, automation or application we are proposing engineering work for.

Whereas the why for an issue would focus on what we would deliver to team members, the problem or goal would focus on which data sources, features or capabilities need to be added or changed in specific tools or infrastructure maintained by Vulnerability Management to effect the desired outcome.

For example: When delivering a new feature, the why for the issue might describe a new piece of data other team members have asked for. In this situation, the problem or goal would describe the limitations of the current implementation and why it cannot provide, store or represent the required information, and what needs to be delivered. Something like: "With the current provider implementation, we do not have access to vendor CVSS data for the relevant Linux

distributions. In order to deliver the requested functionality, we need to add support for processing Alpine and Chainguard advisory data as part of the existing advisory processing code."

A good description of the problem or goal should reference the business problem, document the current technical limitation, and then propose the MVC to product the required outcome.

Documenting a high level plan

We must document, at a high level, the work covered by an Engineering Issue. This is essentially a very light-weight solution proposal and design exercise, where during documenting an issue, we work to understand current limitations, the problem being solved, the goals of the issue, so we can document the high level technical changes which are required to be performed to achieve the stated outcome.

This is less about defining the problem, and more about breaking down the problem into a series of MVCs which will become the scope of the issue. We must be careful to consider other work required in other projects, or infrastructure changes required, and to ensure those issues are also linked as dependencies as part of the high level plan.

It should be clear when reading an Engineering Issue which parts of the code will be modified, what the intended changes to those components are, and exactly which high level steps will be required.

This helps to document engineering work, helps to validate sizing, helps to ensure we have understood and considered a suggested implementation, helps to reduce scope creep, and makes it easier to collaborate and hand off issues to other engineers if necessary.

Working on issues

Once an issue has been included in a milestone, and assigned to a team member, that team member must ensure that the issue is worked on during the milestone, and kept up to date with progress.

Currently, we use the issue status to reflect completion status of issues, however we are planning to introduce workflow progress labels for issues to assist in identifying which issues are being planned, are planned, are being worked on currently, are blocked or are complete. Currently, team members must ensure the description reflects the current status, and issues are closed once implemented.

Where scope changes during implementation, for example when it becomes clear additional work is required to meet the goals of an issue, the additional work should be documented in a new engineering issue, which should be linked to the current in-progress issue. It should then be included the planning for the next milestone, noting in the current issue that additional work is required to deliver the stated goal.

Where the priority or other time-related factors change during a milestone due to external factors, we should update existing issues which are currently being worked on. If an issue needs to be worked on due to these factors and it was not included in the milestone plan, the Engineering Manager and Staff Engineer in the team should be notified on the issue so a

decision can be made. If appropriate, the milestone plan will be updated to note the deviation from the plan, and include the new work. Use your judgement, and only wait for a determination about including an issue in the milestone plan from the Engineering Manager or via a +1 from a Staff Engineer if doing so will not impact operations or the operations of other teams, and note the policy exceptions set out in this handbook page.

Writing Code or Configuration

This policy sets out the practices we must follow when writing or modifying code maintained by Vulnerability Management. This policy is intentionally very light on requirements, and is focused on individual team member productivity and removing barriers to contribution and inefficiency.

This policy only applies to contributing code to projects maintained by Vulnerability Management. Projects outside of Vulnerability Management will have their own standards which should be adhered to if required.

Language choice

We have a strong preference for writing code in Go as Vulnerability Management team members are familiar with it, it has a good ecosystem of libraries and modules we can reuse, we have a lot of existing code to draw on or extend, and the tools are very opinionated (batteries are included) so we do not need to make very many specific tooling decisions. Additionally, Go is well supported by GitLab CI/CD and Security scanning features, so we can use the GitLab product effectively during development.

Where it makes sense to due collaboration with other teams, we may select different languages such as Ruby to aid in contributing code or demonstrating ideas for teams which are more familiar with Ruby and/or Rails. For any web-based applications we build, we must also use Ruby on Rails as soon as a web frontend is required - Web development beyond simple status pages and APIs in Go is inefficient and cumbersome due to a lack of high level frameworks. Coding web-facing features in Rails also helps us to contribute back to the GitLab product as the paradigms in our code will translate more easily into the GitLab Rails codebase than if we had written them in Go.

Writing automation in Python or Bash scripting should be avoided as much as possible. Node.js is used by one project we maintain, but we must not write any new tooling in Node.js due to difficulty of writing robust tooling in this language, and general team unfamiliarity with the ecosystem.

Code style and linting

We must include appropriate code linting in the CI configuration for our projects. The rules used should not be custom, unless customization can be justified and planned as engineering work with demonstrable benefit to team efficiency or maintainability.

When enabling linters for a project, the most common linter which is considered good practice within the community of the predominant programming language (or languages) used in the project should be enabled.

Some good examples:

- Go: We should enable golangci-lint, gofmt and go vet. We should validate base linting rules. Go is opinionated and standard linting rules are fairly consistent.
- Python: We should enable linting which honours PEP8. A combination of pylin and black is a good choice.
- Ruby: rubocop is a good choice.
- Bash: shellcheck is a good choice, especially for catching common mistakes.

Security scanning

Our projects must have appropriate security scanning enabled.

- For infrastructure as code (such as Terraform), we must enable IaC Scanning.
- For Go, Ruby and general source code projects, we must enable SAST Scanning
- For all projects, we must enable Dependency Scanning, and Secret Detection with Secret Push Protection
- For projects which build containers, we must enable Container Scanning

IDE Configuration

Team members must ensure their IDE configuration is consistent with the guidelines in this standard, and also consistent with the linting rules we have configured for our projects, which use standard rule sets for the appropriate language.

Mis-configured IDEs will automatically introduce unrelated formatting and import ordering changes into MRs which make them harder to review, introduce formatting churn between team members who collaborate on the same code, and may lead to you needing to rework your MRs consistently to remove those changes.

If you would like to use or suggest a specific automatic formatting feature of your IDE, you must first propose changes to this standard and the linting configuration of related projects, demonstrating the value of the linting rules you would like to introduce and documenting them as a custom linting rule other team members should configure in their IDE. Once a custom rule has been merged and documented, you can configure your IDE to apply that formatting automatically.

Dependency management

We must enable automation to keep dependencies up to date, or create issues for regular dependency updates for maintained projects. Tools like Renovate can automate and greatly improve the efficiency of dependency updates and reduce the time between new dependencies being released and MRs being created to include them in code we maintain.

At this stage, there is not yet an approved way to use Renovate with our projects, we have (private) engineering issues (vulnmapper, vminfra) open to explore and set up Renovate for Vulnerability Management projects. Once that is available, usage of Renovate will be required for all projects.

Code Review

The following conditions must be met for a merge request to be approved and merged. Each part of the policy has a header and includes examples. As someone writing code or reviewing code in the Vulnerability Management team, you must be familiar with this policy. For team members in other teams, this is provided as a reference for how maintainers in Vulnerability Management will be reviewing any submissions you make.

Automated tests must pass

All automated tests and code hygiene tests should pass. If there are exceptions, these should be noted on the MR and reviewed as part of MR review.

For example, an exception to this might be an MR to fix a unit test which is failing, in which case the failing test which is addressed by the MR may not pass until the changes are merged and deployed.

Merge Requests must be reviewed prior to merging

Any contribution must be reviewed by at least one other Vulnerability Management team member, unless you are working on an experiment or project with no interaction or impact to Vulnerability Management tooling, infrastructure or operations in general.

For example: A time when you might not need to solicit a review, is when you are working on a personal experiment such as an entirely new security scanning tool, which does not touch GitLab data and does not interfere in any way with GitLab team members or operations. Before this tool can enter general usage by the team or other teams at GitLab however, it should have a code review in line with this policy. Once validated, it would be promoted to a Vulnerability Management project, and would need to adhere to this policy.

For example: if you were creating a tool to test out if you could scan a new type of artefact we don't currently support, and the tool operated on images or packages without making or publishing any changes, just to validate the concept prior to further engineering work.

In case of changes which directly impact and require input from another team, we should solicit feedback directly on the MR from a member of that team. An example of this would be enabling a VulnMapper feature which would configure VulnMapper to operate on issues or vulnerabilities which another team is responsible for. This is designed to aid in collaboration and build communication with impacted teams into our general awareness and processes.

Reviews must focus on desired outcomes

As part of documenting and sizing any work included in a milestone, we require that issues document the approach, estimate the effort required, and document the reasons why the change or new functionality is required. The reasons a change or new feature is required must be reviewed against the anticipated outcomes of approving and merging a merge request.

For example: If an issue included in the current milestone is describing a specific new field to introduce from an external data source, we should verify to the best of our ability (with tests, if appropriate) that the field is introduced by the changes and contains useful, sensible data within the context of the original issue. If it does, that would be a good reason to approve a merge request. If it does not, that would be a reasonable grounds for asking for

changes and withholding approval.

Reviews must focus on maintainability

As a reviewer you must always consider how maintainable a contribution is. You must review contributions and consider if they will add additional maintenance burden (such as dependencies, abstractions, functionality, inability to test changes in the future) not justified by the goal or priority of the changes. It is appropriate to withhold approval so long as you are able to provide viable alternatives which are demonstrably more maintainable. This may take the form of alternative libraries which are better maintained, reducing the complexity of the implementation, removing redundant code or abstractions, or adding tests which help to validate the changes being reviewed function as intended. These suggestions must be in keeping with the rest of this standard, however, and must demonstrate improved maintainability as part of validating the maintainability concern.

Reviews must focus on security

We're a part of GitLab Security, and we specifically manage vulnerabilities in other people's codes and systems. Introducing security vulnerabilities into our own code is ironic and unfortunate, so we should do everything we can to avoid it. We configure appropriate GitLab security scanning features on all of our projects, but if you have demonstrated concerns about the security of a proposed change, you must provide it as review feedback. It is appropriate to withhold approval if an MR introduces additional vulnerabilities based on scanning, or you find a vulnerability in the implementation through manual review. You must review vulnerabilities introduced in an MR using the MR Security widget as part of the security review of an MR, and all new findings should be resolved. False positive findings should be provided as feedback via an issue to the appropriate team under Application Security Testing.

Reviews must not focus on personal preferences

When reviewing code we must be objective about the goals, functionality and appropriate performance levels of code under review. If you feel that you would have implemented something differently without concrete reasons which align with our code review guidelines, you must not withhold approval of an MR stipulating that code be rewritten based on your preferences. You are of course encouraged to demonstrate the reasoning for your preference and discuss it with other team members, but we must not block work based on this.

Personal preference can apply to many areas of coding, including formatting preferences, and small idiomatic preferences. This also sometimes takes the form of "nitpicks" which are very rarely helpful feedback and should be avoided in code review. There is a great post discussing this kind of feedback in depth and with empathy.

Reviews must not focus on style for style's sake

We set reasonable style expectations and aim to include basic, consistent linting throughout our code and expect team members to configure their IDEs appropriately for projects they work on, to make collaboration easy and code consistently readable. We should not review the style of other team member's code if it has passed our automated linting, especially based on personal preference. If the code meets the stated goals and is expected to be appropriately functional and performant based on the defined usage, approval should not be withheld due to

stylistic feedback.

If a member of the team would like to make changes to how we style our code, then they should propose engineering work to modify our automated linting to accommodate the desired style changes, rather than asking for ad hoc changes to merge requests they review. This is more repeatable and consistent and allows us to better consider the necessity and benefits of any style changes.

MRs must not include unrelated changes

If your IDE re-formats the entire file you are working on automatically, or if you bump all of the dependencies of a project when working on, these must not be included in an MR which is making planned and scoped changes to a project. It is appropriate to withhold review and ask for the unnecessary changes to be included in a separate MR as part of planned maintenance work.

Code must not be prematurely optimized

Premature optimization refers to the process of making all code as performant as possible, despite the use case and ignoring the lack of resource constraints or scalability requirements to meet the stated goal of the code.

We must absolutely optimize our code however, when it is required to operate within the constraints of the system based on the goals of the change.

For example: We need to calculate the priority of a GitLab issue based on several factors such as advisory CVSS, known exploitation and vendor scoring information. A merge request introduces a simple helper function which can be called with these values to return the priority as a floating point number. We must not withhold approval of the merge request based on potential optimization of the logic in the function, if it meets the goal of the planned work, unless justified. If the function is being called once a day over 100 records, withholding approval would be inappropriate. If the function is being called 100,000 times every minute and we anticipate this change will introduce significant load and delays in processing, then it is absolutely appropriate to make optimization suggestions and withhold approval.

Code must not introduce unnecessary abstractions

Making code more understandable, and removing repetition of logic in a codebase is an excellent way to reduce overall lines of code, and make maintenance and iteration of code more efficient. However, abstractions of existing or new code must be added only when they can be justified. Unnecessary abstraction can make code harder to understand, lead to additional engineering effort which is not required to solve the stated goal of the MR, and introduce extra work for reviewers. Ideally, abstractions should be added as distinct engineering issues as they become required, and adding them be treated as dependencies for planned work in the current or next milestone which requires the abstraction.

Justifications for adding further abstractions should be provided so that reviewers have the necessary context for why an abstraction is required for the current work being reviewed, or by other work which has also been planned and approved for including in this or the next milestone. This also helps to make sure we have thought out the need for abstraction, and the

level of abstraction as well as required details to meet team goals for a tool or application.

For example: If an issue included in the current milestone requires us to add a new type of information to one of our tools, such as adding information about servers to an existing tools which previously only handled vulnerability detections, we must validate that the minimum viable amount of information is being correctly modelled by the code introduced in the MR, and that any further abstraction is appropriate. If the MR in this example introduced servers, and also generally introduced an further abstraction for cloud assets without concrete justification, that would be grounds for the reviewer asking that the unnecessary abstraction be removed and withholding approval. However, if the need for the abstraction is explained, documented, and required by other planned work, that would be a good reason to approve the changes even with the extra abstraction.

Rule of Three

As a general guideline, the rule of three should be followed). When the same code appears 3 times, it should be refactored into a distinct function, method or similar with appropriate testing and documentation to improve maintainability.

SECTION: security/product-security/vulnerability-management/development-labels.md

Vulnerability Management Engineering uses several sizing labels to indicate how much effort is estimated to be involved in adding a feature or addressing a bug described by an issue. In addition to this, we use weights and milestones. The weights correlate to the labels and this all is designed to make planning out our milestones easy and predictable from a capacity management perspective.

Priority

We use the standard GitLab Engineering Priority Labels to reflect the priority of individual issues. Priority is decided during milestone planning and is informed by company-wide critical projects, customer commitments and Vulnerability Management team priorities. Vulnerability Management team members are expected to follow the priority of issues assigned to them during a milestone and ensure they are working on the highest priority issues first.

Sizing and estimation (Size labels & issue weight)

A single point of weight on an issue roughly represents a single day of engineering work for a single engineer.

For convenience, we also currently apply labels based on weight to give issues a rough size.

Estimated Effort	Label	Weight	
-----	-----	-----	
Less than a day	~vuln-mgmt::Trivial	1	
1-2 days	~vuln-mgmt::Small	2	
2-5 days (1 week)	~vuln-mgmt::Medium	5	
5-10 days (2 weeks)	~vuln-mgmt::Large	10	

| 15-25 days (Most of a milestone) | ~vuln-mgmt::X-Large | 15 |
| 25 days+ (Too large for a milestone) | ~"vuln-mgmt::Too Large" | 25 |

Workflow labels

With consideration of not re-inventing the wheel, the Vulnerability Management team is using workflow labels workflow labels used by Engineering.

These are the subset of label's we use.

Label	How I understand the label
~"workflow::blocked"	Work was started or planned, but is blocked by external factors or another issue
~"workflow::complete"	Work is complete and shipped, issue should be closed
~"workflow::in dev"	Planning done, in current milestone, no blockers, work is in progress
~"workflow::in review"	Planning and dev work complete, blocked on MR review
~"workflow::planning breakdown"	Issue needs to be sized, scoped and documented for potential milestone inclusion
~"workflow::scheduling"	Planning, sizing, etc is done, issue has not been started, but is ready for discussion at the next milestone planning meeting

SECTION: security/product-security/vulnerability-management/encryption-policy.md

Purpose

This policy is intended to outline the encryption controls and requirements at GitLab.

Scope

This policy is applicable to the production environment and any end user devices that store such data. This also includes the GitLab Dedicated single-tenant SaaS offering.

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for following the requirements in this policy
Business or System Owners	Alignment to this policy and any related standards
Product Security Team	Maintain this Encryption Policy and associated standards
Security Management (Code Owners)	Responsible for approving significant changes and exceptions to this policy

Policy

Encryption

Customer data is encrypted at rest. (SC-28)

Corporate owned endpoints are encrypted at rest. (SC-28)

Customer data is encrypted in transit. (SC-8)

Standard

Encryption at GitLab is performed in accordance with GitLab's Encryption Standard and Cryptographic Standard

Exceptions

Exceptions to this procedure will be tracked as per the Security and Technology Policy Exception Management Process.

SECTION: security/product-security/vulnerability-management/incident-response-guidance.md

To provide guidance and insight into our incident response process. Incident response is a key aspect of GitLab's overall security program. This guidance will provide all in scope individuals the information they need to help GitLab ensure incidents are reported, investigated and handled in such a way that minimize security events or data loss.

Definition

The definition of an incident is the first step in determining how to report an incident.

- Security Team Incident: Any violation, or threat of violation, of GitLab security, acceptable use or other relevant policies.

- Infrastructure Team Incident: Anomalous conditions that result in, or may lead to, service degradation or outages.

Scope

This guidance is meant to support all GitLab team members, contractors, advisors, contracted parties interacting with GitLab, customers, individual contributors or any external entity that has a need to report an identified or suspected incident.

Workflows

Incidents at GitLab are separated into two workflows depending on the type of incident reported. This guidance provides links to the associated handbook pages that define specific actions or processes from either our Security Team or our Infrastructure Team. Actions from either of these processes are meant to minimize the impact, operationally or financially, of critical business operations.

Process

1. Identification

A. If you are able to determine the type of incident that has been suspected or identified, report your incident to either Security or Infrastructure.

- Note: GitLab takes any and all incidents seriously. If you are uncertain who to report an incident to, please report your incident using the support web form and your incident report will be internally forwarded accordingly.

2. Reporting Incidents

A. Security:

- Internally or externally through various pathways
- Internally only via Slack or Email

B. Infrastructure:

- Externally through the support web form. Incident response times are based on your SLA
- Internally via on-call

3. Coordination

A. Security:

- The Application Security Team uses the triage rotation to coordinate and respond to security incidents.

B. Infrastructure:

- The Reliability Team Engineer on Call is the first person alerted and is generally a Site Reliability Engineer (SRE) that is responsible for coordination and response to infrastructure related incidents.

4. Containment

A. Security:

- Issue triage
- HackerOne triage

B. Infrastructure:

- Runbooks

5. Remediation and Recovery

A. Security:

- Due dates
- Managing the flow of information via Communication

B. Infrastructure:

- Status updates
- Managing the flow of information via Communication

6. Resolution

A. Security:

- Root Cause Analysis

B. Infrastructure:

- Incident Review

Additional Resources

- Business Continuity Plan
- BU.1.01 - Backup Configuration Control Guidance
- Security Incident Response Guide

SECTION: security/product-security/vulnerability-management/infrastructure-vulnerability-procedure.md

This procedure applies to vulnerabilities identified in GitLab Production infrastructure and ensures implementation of the Vulnerability Management Standard. This procedure is designed to provide insight into our environments, promote healthy patch management among other preventative best-practices, and remediate risk; all with the end goal to better secure our environments and our product.

Scope

Security and Infrastructure partnered to come up with a scope that would make sure all of our critical environments and systems were covered during deployment. The following environments are currently in-scope for GitLab.com and GitLab Dedicated production:

Environment	Project/Account	Production	Deployed
---	----	---	---
GCP	gitlab-production	yes	yes
GCP	gitlab-ops	yes	yes
GCP	gs-production	yes	yes
GCP	env-zero	yes	yes
GCP	gemnasium-production	yes	yes
GCP	service-prod	yes	yes
GCP	gitlab-ci	limited	yes
AWS	gitlab-com	yes	yes
Azure	GitLab	yes	no

>Note: If you believe a system you are responsible for should be included in the vulnerability management process, please contact the Security Vulnerability Management Team.

With these environments scoped out and Wiz connectors ready, we can begin the vulnerability management process. Keep in mind that vulnerability management is a feedback loop - vulnerability detection provides the vulnerability data which is analyzed and ingested to mitigate and remediate found vulnerabilities. Feedback from this process feeds into preventative initiatives that further secure our environments.

Application layer vulnerabilities are our of scope and managed via the Application Vulnerability Management Procedure.

Roles & Responsibilities

| Role | Responsibility |

| ----| -----|

| Vulnerability Management Team | Responsible for implementing and maintaining this Vulnerability Management Procedure |

| Infrastructure | Responsible for remediating vulnerabilities according to procedure |

| Security Leadership | Responsible for making final determination on risk acceptance (exceptions) |

Procedure

Currently, we break down vulnerability management into the following steps:

1. Vulnerability detection

This step is where we run detection workloads against our environments to identify vulnerabilities. Once the connector is configured, workloads run on regular cadences that meet or exceed our compliance framework requirements. The vulnerability remediation SLA timeframes begin as soon as a vulnerability is discovered by a workload

2. Ingestion/Auto Triage

Vulnerability findings are pulled from Wiz.io into VulnMapper utilizing the wiz.io provider, this provider provides a degree of normalization and stores vulnerability findings within it's local DB for other providers to consume.

The gitlab-provider in VulnMapper does the heavy lifting between VulnMapper and GitLab. It raises both a vulnerability and linked vulnerability tracking issue, performs auto triage and state syncing into GitLab.

Note: Vulnerability creation is ready, but disabled due to Current GitLab GraphQL constraints.

vulnerabilities and tracking issues project

Vulnerability remediation issues should be tagged with the vulnerability type label. These leverage GitLab's scoped label capability. The following labels exist to track the vulnerability remediation workflow:

- ~vulnerability::vulnerable: This label identifies that the vulnerability has been opened, but not validated and is considered impactful to our environments per the assigned priority label. With this label a vulnerability issue should not be closed.
- ~vulnerability::validated: This label identifies that the vulnerability has been validated as legitimate and is scheduled for mitigation or remediation. With this label a vulnerability issue should not be closed.
- ~vulnerability::falsepositive: This label identifies that the vulnerability has been validated as a false positive and is no longer impactful to our environments. With this label a vulnerability issue can be closed.
- ~vulnerability::exception: This label identifies that the vulnerability has been validated as legitimate and has an approved exception issue to account for a business need. In extreme circumstances, a vulnerability issue can be closed with an exception.
- ~vulnerability::mitigated: This label identifies that the vulnerability has been validated and triaged. The impact has been reduced through compensating controls, but not remediated (it is still actively identified on vulnerability scans). With this label a vulnerability issue should not be closed.
- ~vulnerability::remediated: This label identifies that the vulnerability has been remediated and the remediation has been validated. With this label a vulnerability issue can be closed.

We also add the VM label to all Vulnerability issues to scope the issues in the Vulnerability Management issue board.

3. Validation

Validation is an important part of vulnerability management. This is where we investigate to ensure that the vulnerability being reported has properly been identified.

Vulnerabilities can sometimes be identified during a workload, but are not actually on the system. This can happen for a number of reasons, but most commonly is the result of incorrectly flagged ports or services. These are classified as false positives and would go through the process to be closed as a false positive.

4. Remediation

Remediation is the part of the process in which a validated vulnerability is fixed. The remediation process would be tracked in the corresponding tracking issue raised here. SLAs are in place to help prioritize vulnerability based on severity. The vulnerability remediation SLA timeframes begin as soon as a vulnerability is detected by a workload. Once a vulnerability is remediated, Wiz.io will mark the vulnerability as resolved, this will flow thorough the normal ingestion path and be synced into GitLab.

We've implementing an escalation path for remediation issues in the Infrastructure issue tracker that automatically tags the VM DRI and backup when remediation issues are approaching/past SLAs.

For improved tracking of remediation issues, we are using GitLab Epics. The remediation epic includes monthly subepics that track remediation progress for that month. If remediation SLAs do not require a vulnerability to be remediated in a month, it will be rolled over into the following subepic until remediated or its due date passes.

Vulnerability Issue Workflows

There are several ways a vulnerability issue can be closed - below are some common vulnerability workflows using the vulnerability labels as reference:

Closed as Remediated

The most common workflow is to close a vulnerability issue as Remediated. This means that a vulnerability has been validated and remediation has taken place. Below is the workflow:

```
mermaid
graph TB
    SubGraph1 --> Node2
    subgraph "Vulnerability Issue Status: Closed"
        Node2(Vulnerability:Remediated)
    end

    subgraph "Vulnerability Issue Status: Open"
        Node1[Vulnerability:Vulnerable] --> SubGraph1[Vulnerability:Validated]
    end
end
```

Closed as False Positive

A vulnerability must always be validated - but sometimes the validation can prove that a vulnerability is a false positive. Below is the workflow:

```
mermaid
graph TB
    SubGraph1 --> Node2
    subgraph "Vulnerability Issue Status: Closed"
        Node2(Vulnerability:FalsePositive)
    end

    subgraph "Vulnerability Issue Status: Open"
        Node1[Vulnerability:Vulnerable] --> SubGraph1[Vulnerability:Validated]
    end
end
```

Closed as Exception

Sometimes issues arise that would otherwise prevent a vulnerability from being remediated or mitigated. While commonly, these would result in an open Exception vulnerability issue status, there are unique cases where an issue can be closed as an exception. Below is the workflow:

```
mermaid
graph TB
```

```
SubGraph1 --> Node2
subgraph "Vulnerability Issue Status: Closed"
Node2(Vulnerability:Exception)
end
```

```
subgraph "Vulnerability Issue Status: Open"
Node1[Vulnerability:Vulnerable] --> SubGraph1[Vulnerability:Validated]
end
```

Open as Exception

Closed issue via the Exception process are very rare. Generally, an exception is a non-permanent way to assume risk on a vulnerability due to extenuating circumstances in which remediation can not take place within the required SLAs. Below is the described the workflow:

```
mermaid
graph TB
```

```
SubGraph1 --> Node3
subgraph "Vulnerability Issue Status: Closed"
Node3(Vulnerability:Remediated)
end
```

```
subgraph "Vulnerability Issue Status: Open"
Node1[Vulnerability:Vulnerable] --> Node2(Vulnerability:Validated) -->
SubGraph1[Vulnerability:Exception]
end
```

Open as Mitigated

Another common workflow is when a vulnerability is validated and a fix is scheduled for some time in the future (within the SLA). If we're able to, we will put mitigation in place in the interim to reduce the risk from the vulnerability. Below is the described workflow:

```
mermaid
graph TB
```

```
SubGraph1 --> Node3
subgraph "Vulnerability Issue Status: Closed"
Node3(Vulnerability:Remediated)
end
```

```
subgraph "Vulnerability Issue Status: Open"
Node1[Vulnerability:Vulnerable] --> Node2(Vulnerability:Validated) -->
SubGraph1[Vulnerability:Mitigated]
end
```

6. Feedback

The last step is for the Vulnerability Management Team, Security Incident Response Team and Infrastructure to determine what we can learn from each vulnerability remediated. This may be an improvement on the vulnerability management process itself or establishing preventive mechanisms for a repetitive vulnerability type. This feedback will be documented in the vulnerability issue and could result in additional issues being opened.

As stated above, this process is a cyclical loop. Vulnerability detections are recurring, providing new vulnerability data that feed new vulnerability remediation and exception issues which then help update/escalate open issues/processes.

Vulnerability detection workload Schedule

Vulnerability detection workloads occur on a less than daily basis in our scoped environments. There is no configured schedule and this should be considered as continuous detection.

{{% alert title="Note" color="primary" %}}

For more information, please visit the gsecurityvulnmgmtnotifications channel on Slack where there are links to reference documentation.

{{% /alert %}}

Exceptions

Refer to Vulnerability Management Standard Risk Acceptance (Exceptions)

References

- Vulnerability Management Team
- Vulnerability Management Standard
- Application Vulnerability Management Procedure

SECTION: security/product-security/vulnerability-management/labels.md

A standard set of vulnerability management labels are used across GitLab to assist team members in handling security vulnerabilities in GitLab-maintained projects.

These labels are applied automatically by automation managed by the Vulnerability Management team. Automated labeling happens on a daily cadence. Several data sources are used to enrich existing security tracking issues, and are labels are designed to increase the amount of information and context available to team members about vulnerabilities present in the projects they maintain. The primary goal is to reduce the amount of manual research and triage work required, and help to apply consistent and effective management of vulnerabilities across GitLab.

Tracking Issue Labels

GitLab is used to track vulnerabilities as both vulnerabilities and issues, across GitLab's infrastructure and projects. As part of this process, various standard labels are applied to

track the status of managing vulnerabilities through to mitigation and remediation.

Remediation SLA Labels

The labels used to indicate the SLA which applies to vulnerability findings tracked as issues are documented on the Vulnerability Management SLAs page.

Vulnerability Information and Workflow Labels

The following labels are currently used, or are available for use, to annotate and enrich security tracking issues.

Label	Further Reading	Purpose
-----	-----	-----
security	Security Handbook	Security and Engineering groups use this label to indicate an issue relates to a security tracking issue
bug::vulnerability	Composition Analysis Handbook, AppSec Vulnerability Management Process	Used to represent a security tracking issue with a vulnerable software component
OS::<x> <y>		Indicates the detected OS vendor and release of a scanned artifact related to a vulnerable software component
Vulnerability Management::Vendor Severity Deviation		Indicates the vendor security advisory severity was different to the severity of a detected vulnerability in the GitLab scan artifacts or vulnerability report entry
Vulnerability Management::Vendor CVSS Deviation		Indicates the vendor designated CVSS score has either been calculated or reported differently to the score reported by the security scanner which generated the vulnerability report entry in GitLab
Vulnerability Management::Process Deviation	Vulnerability Management standard	Indicates the GitLab Vulnerability Management standard has not been followed for this issue (i.e., a tracking issue has been closed when a vulnerability is still detected)
Vulnerability Management::Vulnerability Mismatch		Indicates the detected and linked vulnerability was found to not match the tracking issue based on OS, CVE or other vulnerability data
Vulnerability::Vendor Package::Fix Available		An updated vendor package is available which fixes this vulnerability
Vulnerability::Vendor Package::Fix Unavailable		There is no vendor fix for this vulnerability, but there may be in the future, when published by the vendor
Vulnerability::Vendor Package::Will Not Be Fixed		There will not be a fix made available for the vulnerable software package (typically because the vulnerability is not applicable)
Vulnerability::Vendor Base Container::Fix Available		An updated vendor base container image is available which fixes this vulnerability
Vulnerability::Vendor Base Container::Fix Unavailable		There is no updated base container image fixing this vulnerability, but there may be in the future, when published by the vendor
Vulnerability::Vendor Base Container::Will Not Be Fixed		There will not be a fixed container made available for the vulnerable container image (typically because the vulnerability is not applicable)
Vulnerability Impact::Scan Severity::[High/Medium/Low]		The severity reported by the vulnerability as detected by the scanner which detected the vulnerability
Vulnerability Impact::Scan CVSS::(x.x)		The CVSS (which defaults to CVSSv3, and falls back to CVSSv2 if v3 is not available) reported by the vulnerability scanner which detected the

vulnerability |

| Vulnerability Impact::Vendor Severity::[High/Medium/Low] | | The severity reported by the most relevant software vendor (package or container), if the vendor was detected |

| Vulnerability Impact::Vendor CVSS::(x.y) | | The CVSS (which defaults to CVSSv3, and falls back to CVSSv2 if v3 is not available) reported by the most applicable software vendor (package or container), if the vendor was detected |

| Vulnerability Management::Pending NVD Analysis || Indicates if CVE is awaiting analysis by NVD |

| Vulnerability Impact::Public Exploitation::Exploited | CISA Known Exploited Vulnerabilities List | Indicates the vulnerability is known to have been publicly exploited. Currently CISA is the only data source, but other data sources can be added in the future. |

| Vulnerability Impact::Public Exploitation::Unknown | CISA Known Exploited Vulnerabilities List | Indicates the vulnerability is not found on the CISA KEV list. This does not necessarily mean that the vulnerability is not or has not been under active exploitation, simply that CISA has not been provided with evidence of exploitation. |

SECTION: security/product-security/vulnerability-management/lifecycle.md

Tracking Issue Lifecycle

The typical lifecycle of a vulnerability detection at GitLab, at a high level, follows the following steps:

1. Vulnerability detected by scanner or otherwise reported
1. A GitLab issue is created to track the vulnerability
1. The vulnerability is remediated within SLA
1. The issue is closed

Depending on the environment and what is being scanned, the tracking issue will be handled by different GitLab team members.

Issues may also be enriched by different automated tools, such as (VulnMapper, which assist with streamlining the detection and remediation process.

When an issue cannot be remediated within SLA, additional procedures exist for adjusting or exempting vulnerabilities from SLA. See the section in this page on Risk Acceptance & SLA Exception for detail.

Closing tracking issues

To reduce the number of non-actionable issues being tracked by development groups, we've looked at the various situations where issues can be safely closed, without compromising our ability to react to changes in fix availability or mitigating factors.

Vulnerability tracking issues can be closed when:

- A vulnerable dependency or package has been updated, and an updated software artifact (container image, package) has been shipped
- A vendor dependency in non-FedRAMP container images will not be fixed by the software vendor (i.e. Debian) as it is not impactful

- A vendor dependency in a FedRAMP container image will not be fixed by the software vendor (i.e. Red Hat) as it is not impactful, and a DR has been opened and linked to the tracking issue
- There is a false positive detection in a non-FedRAMP container image or project and an exception issue has been created or updated and linked to the tracking issue
- There is a false positive detection in a FedRAMP container image and a DR has been opened or updated and linked to the tracking issue

Example vulnerability lifecycles

An impactful vulnerable package is detected by Wiz.io on a GitLab-managed infrastructure system

1. A vulnerability and tracking issue pair is created by VulnMapper
1. The issue is remediated as part of GitLab's production patching cycle
1. The issue is closed

An impactful vulnerable package is detected in a non-FedRAMP GitLab container image

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The vulnerable package is updated
1. An updated image is shipped
1. The issue is closed

An vulnerable package is detected in a non-FedRAMP GitLab image, no fix available

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. No fix will be made available by the vendor, as indicated by the automated issue labels or manual research
1. The linked vulnerability finding from the container scan report is dismissed
1. The issue is closed

An impactful vulnerability is detected in a non-FedRAMP GitLab image, fix is available

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. No fix will be made available by the vendor, as indicated by the automated issue labels or manual research
1. The linked vulnerability finding from the container scan report is dismissed
1. The issue is closed

An impactful vulnerability is found in a dependency used by a GitLab software project

1. A dependency scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The dependency is updated
1. An updated version of the software is shipped
1. The issue is closed

An impactful, unfixed vulnerability is found in a dependency used by a GitLab project

1. A dependency scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The responsible development group works with the library maintainer or GitLab owner of the dependency to understand fix availability
1. The maintainer(s) indicates a fix will not be available with SLA
1. Mitigating controls are explored to either validate or mitigate the impact of the vulnerability
1. Mitigating controls or switching to an alternate library can not be implemented within SLA
1. An exception issue is opened detailing the options explored to extend the SLA as needed
1. An updated version of the software is shipped with either a fix or mitigation in place
1. The issue is closed

A vulnerability which won't be fixed is found in a dependency used by a GitLab project

1. A scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The responsible development group works with the library maintainer or GitLab owner of the dependency to understand fix availability
1. Either there is no response, or the project is not maintained, or the vulnerability has been rejected and won't be fixed for any reason
1. Mitigating controls are explored to either validate or mitigate the impact of the vulnerability
1. Mitigating controls or switching to an alternate library can not be implemented within SLA
1. An exception issue is opened detailing the options explored to extend the SLA as needed
1. An updated version of the software is shipped with either a fix or mitigation in place
1. The issue is closed

An impactful vulnerable package is detected

in a GitLab FedRAMP container image

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The issue is automatically labelled by VulnMapper with the appropriate Vulnerability Management Labels based on fix availability data published by Red Hat
1. A fix is available as indicated by the labels, and the responsible development group updates the image with the fixed package
1. The issue is closed once the updated image is shipped to FedRAMP environments

A vulnerable package which the vendor has decided will not be fixed is detected

in a GitLab FedRAMP container image

1. A container scan finding is created in GitLab
1. An issue is created to track remediation of the finding
1. The issue is automatically labelled by VulnMapper with the appropriate Vulnerability Management Labels based on fix availability data published by Red Hat
1. If a fix will not be made available by the vendor as indicated by labels or manual research,

the Deviation Request documentation should be reviewed to see if a Deviation Request should be appropriate to adjust the SLA for remediation.

1. A DR is c